

Network Devices and TCP/IP Analysis

1. Study about the functionality of intermediate devices used in the Internet and compare it.

Modem

A modem (Modulator-Demodulator) is a device used to convert digital signals from a computer into analog signals for transmission over communication lines, and vice versa. It enables internet connectivity by allowing digital devices to communicate over telephone or cable networks. Without a modem, direct communication over such media would not be possible.

Hub

A hub is a basic networking device that connects multiple devices in a local area network (LAN). It operates at the physical layer and simply broadcasts incoming data packets to all connected devices, regardless of the intended recipient. This leads to unnecessary traffic and inefficiency, making hubs largely obsolete in modern networks.

Repeater

A repeater is used to regenerate and amplify weak signals to extend the transmission distance in a network. It operates at the physical layer and does not interpret the data, but simply restores the signal strength to prevent loss or degradation over long distances.

Bridge

A bridge connects two or more LAN segments and operates at the data link layer. It filters network traffic by using MAC addresses to determine whether to forward or block data frames. This helps reduce unnecessary traffic and improves network efficiency compared to hubs.

Switch

A switch is an advanced version of a bridge that connects multiple devices within a LAN. It operates at the data link layer and uses MAC addresses to forward data only to the intended device. This significantly reduces network congestion and improves performance, making switches widely used in modern networks.

Router

A router is a networking device that connects different networks and forwards data packets between them using IP addresses. Operating at the network layer, it determines the

best path for data transmission. Routers are essential for internet communication as they connect local networks to external networks.

Firewall

A firewall is a security device that monitors and controls incoming and outgoing network traffic based on predefined rules. It can be hardware or software-based and helps protect networks from unauthorized access, attacks, and malicious activities by filtering packets.

Gateway

A gateway is a device that connects networks using different protocols and architectures. It acts as a translator, converting data formats so that communication between dissimilar systems is possible. Gateways operate across multiple layers of the OSI model and are essential in heterogeneous network environments.

2. Study about UNIX commands used in TCP/IP.

1. ftp (File Transfer Protocol)

Used to transfer files between systems

```
ftp <hostname>
```

2. host

Resolves domain names to IP addresses

```
host google.com
```

3. ipconfig (Windows) / ip (Linux alternative)

Displays network configuration

```
ip addr
```

4. netstat

Displays network connections, routing tables, statistics

```
netstat -a
```

5. ip

Advanced tool for network configuration

```
ip route
```

6. ping

Checks connectivity between two devices

```
ping google.com
```

7. route

Shows or modifies routing table

```
route -n
```

8. scp (Secure Copy)

Secure file transfer over SSH

```
scp file.txt user@host:/path
```

9. sftp (Secure FTP)

Secure file transfer session

```
sftp user@host
```

3. Write a program to implement CRC algorithms and find the CRC Code. Consider the data = 101110 and divisor polynomial $x^3 + 1$.

```
#include <iostream>
#include <string>
using namespace std;

string xorDivide(string data, string divisor)
{
    int n = divisor.length();

    for (int i = 0; i <= data.length() - n; i++)
        if (data[i] == '1')
            for (int j = 0; j < n; j++)
                data[i + j] = (data[i + j] == divisor[j]) ? '0' : '1';
    return data.substr(data.length() - (n - 1));
}

int main()
{
    string data = "101110";
    string divisor = "1001"; //  $x^3 + 1$ 

    int degree = divisor.length() - 1;
    string appendedData = data + string(degree, '0');
    string crc = xorDivide(appendedData, divisor);
```

```
string codeword = data + crc;

cout << "Data: " << data << endl;
cout << "Divisor: " << divisor << endl;
cout << "CRC: " << crc << endl;
cout << "CRC Codeword: " << codeword << endl;

return 0;
}
```

```
→ cd "/home/sid/work/acads/
Data: 101110
Divisor: 1001
CRC: 011
CRC Codeword: 101110011
```

HTTP Protocol Analysis

Q1: Answer the following questions for captured file `http.pcap`

1. List 3 different protocols in the protocol column

TCP, HTTP, and DNS are the three different protocols that appear in the protocol column in the unfiltered packet-listing window.

2. Time taken between HTTP GET and HTTP OK response

- Selected packet **217** for HTTP GET request
- Selected packet **219** for HTTP OK response

Timestamps:

- Packet 217 (GET) → 12:49:01.078947
- Packet 219 (OK) → 12:49:01.271363

Time Taken:

12:49:01.271363 - 12:49:01.078947 = 0.192416 seconds

This value is also shown in the “**Time since request**” field in the HTTP section of the OK packet.

3. Internet Addresses

- Internet address of `iitd.ac.in` → **103.27.9.167**
- Internet address of my computer → **127.0.1.1**

4. HTTP Messages (GET and OK)

The HTTP GET request and HTTP OK response messages were extracted and printed.
(Attached separately in PDF as per submission requirement.)

5. File Download Details

- **File Name:** IITD-IRD-122-2017.pdf
- **Packet Number (GET request):** 478
- **File Size:** 18897 bytes
- **Download Completion Time:**

Q2: Program to Process CSV File

Python Code

```
import pandas as pd
import re

# Read CSV file
df = pd.read_csv("http.csv")

# Regex for extracting ports
port_pattern = re.compile(r'(\d+)\s*>\s*(\d+)')

# HTTP keywords
http_keywords = ["GET", "POST", "HEAD", "PUT", "DELETE", "HTTP/1.1"]

with open("output.txt", "w") as f:
    # Header
    header = (
        f"{'Source IP':<20}"
        f"{'Destination IP':<20}"
        f"{'Source Port':<15}"
        f"{'Destination Port':<20}"
        f"{'HTTP Request / Response':<40}\n"
    )
    f.write(header)
    f.write("=" * 115 + "\n")

    for _, row in df.iterrows():
        src_ip = str(row['Source'])
        dst_ip = str(row['Destination'])

        src_port = "-"
        dst_port = "-"
        http_info = "-"

        # Extract ports
        match = port_pattern.search(str(row['Info']))
        if match:
            src_port, dst_port = match.groups()

        info = str(row['Info'])
        protocol = str(row['Protocol'])

        # Extract HTTP messages
        if protocol == "HTTP" or any(k in info for k in http_keywords):
            http_info = info

        # Write formatted output
        f.write(
            f"{'src_ip':<20}"
```

```

        f"{dst_ip:<20}"
        f"{src_port:<15}"
        f"{dst_port:<20}"
        f"{http_info:<40}\n"
    )

    print("Output saved to output.txt")

```

Output

The output is saved in `output.txt`, which contains:

- Source IP
- Destination IP
- Source Port
- Destination Port
- HTTP request/response messages

Source IP	Destination IP	Source Port	Destination Port	HTTP Request / Response
=====				
XiaomiCommun_9e:9c:c3Broadcast	-	-	-	-
HonHaiPrecis_8c:90:55XiaomiCommun_9e:9c:c3-	-	-	-	-
192.168.43.153	103.27.9.20	34573	80	-
192.168.43.153	192.168.43.1	-	-	-
192.168.43.1	192.168.43.153	-	-	-
192.168.43.153	103.27.9.20	34574	80	-
103.27.9.20	192.168.43.153	80	34574	-
192.168.43.153	103.27.9.20	34574	80	-
192.168.43.153	103.27.9.20	-	-	GET / HTTP/1.1
103.27.9.20	192.168.43.153	80	34573	-
192.168.43.153	103.27.9.20	34573	80	-
103.27.9.20	192.168.43.153	80	34574	-
103.27.9.20	192.168.43.153	80	34574	-
192.168.43.153	103.27.9.20	34574	80	-
103.27.9.20	192.168.43.153	80	34574	-
192.168.43.153	103.27.9.20	34574	80	-
103.27.9.20	192.168.43.153	80	34574	-
192.168.43.153	103.27.9.20	34574	80	-
192.168.43.153	103.27.9.20	-	-	GET /sites/default/files/ctools/css/2d4fe1c1412881a73110e50a115127de.css?Y HTTP/1.1
192.168.43.153	103.27.9.20	34575	80	-
103.27.9.20	192.168.43.153	80	34574	-
192.168.43.153	103.27.9.20	34574	80	-
192.168.43.153	103.27.9.20	34576	80	-
192.168.43.153	103.27.9.20	34577	80	-
192.168.43.153	103.27.9.20	34578	80	-
103.27.9.20	192.168.43.153	80	34574	-
192.168.43.153	103.27.9.20	34574	80	-
103.27.9.20	192.168.43.153	80	34574	-
192.168.43.153	103.27.9.20	34574	80	-
103.27.9.20	192.168.43.153	-	-	HTTP/1.1 200 OK (text/html)
192.168.43.153	103.27.9.20	34574	80	-

TCP Protocol Analysis

Q1: Client IP Address and TCP Port Number

The IP address and TCP port number used by the client computer (source) to transfer the file to **gaia.cs.umass.edu** are:

- **IP Address:** 192.168.1.102
- **TCP Port Number:** 1161

Q2: Server IP Address and Port Number

The IP address of **gaia.cs.umass.edu** and the port number used for sending and receiving TCP segments are:

- **IP Address:** 128.119.245.12
- **Port Number:** 80

Q3: Client IP and Port (Reconfirmation)

The IP address and TCP port number used by the client computer are:

- **IP Address:** 192.168.1.102
- **Port Number:** 1161

Q4: TCP SYN Segment Details

The sequence number of the TCP SYN segment used to initiate the connection is:

- **Relative Sequence Number:** 0
- **Raw Sequence Number:** 232129012

The segment is identified as a SYN segment by:

- **SYN = 1**
- **ACK = 0**

Q5: TCP SYN-ACK Segment Details

The sequence number of the SYN-ACK segment sent by the server is:

- **Relative Sequence Number:** 0
- **Raw Sequence Number:** 883061785

The acknowledgement number in the SYN-ACK segment is:

- **Relative Acknowledgement Number:** 1
- **Raw Acknowledgement Number:** 232129013

Explanation

The acknowledgement number is calculated by adding **1** to the sequence number of the received SYN segment, since the SYN consumes one sequence number.

Identification

The segment is identified as a SYN-ACK segment by:

- **SYN = 1**
- **ACK = 1**

Q6: Sequence Number of TCP Segment with HTTP POST

The sequence number of the TCP segment containing the HTTP POST command is:

- **Relative Sequence Number:** 164041
- **Raw Sequence Number:** 232293053

Q7: Length of First Six TCP Segments

Considering the TCP segment containing the HTTP POST as the first segment:

Segment	Description	Length (bytes)
1	HTTP POST	50
2	TCP ACK = 162039	0
3	TCP ACK = 162041	0
4	TCP ACK = 162091	0

5	HTTP OK	730
6	TCP ACK = 731	0

Q8: Estimated RTT Calculation

The Estimated Round Trip Time (EstimatedRTT) is calculated using:

$$\text{EstimatedRTT} = (1 - \alpha) \times \text{EstimatedRTT} + \alpha \times \text{SampleRTT}$$

Where:

- $\alpha = 0.125$

Initial Condition

- First SampleRTT ≈ 30 ms
- Initial EstimatedRTT = 30 ms

EstimatedRTT Values

- After 2nd ACK $\rightarrow \approx 35$ ms
- After 3rd ACK $\rightarrow \approx 45.6$ ms
- After 4th ACK $\rightarrow \approx 63.65$ ms
- After 5th ACK $\rightarrow \approx 88.2$ ms

Observation

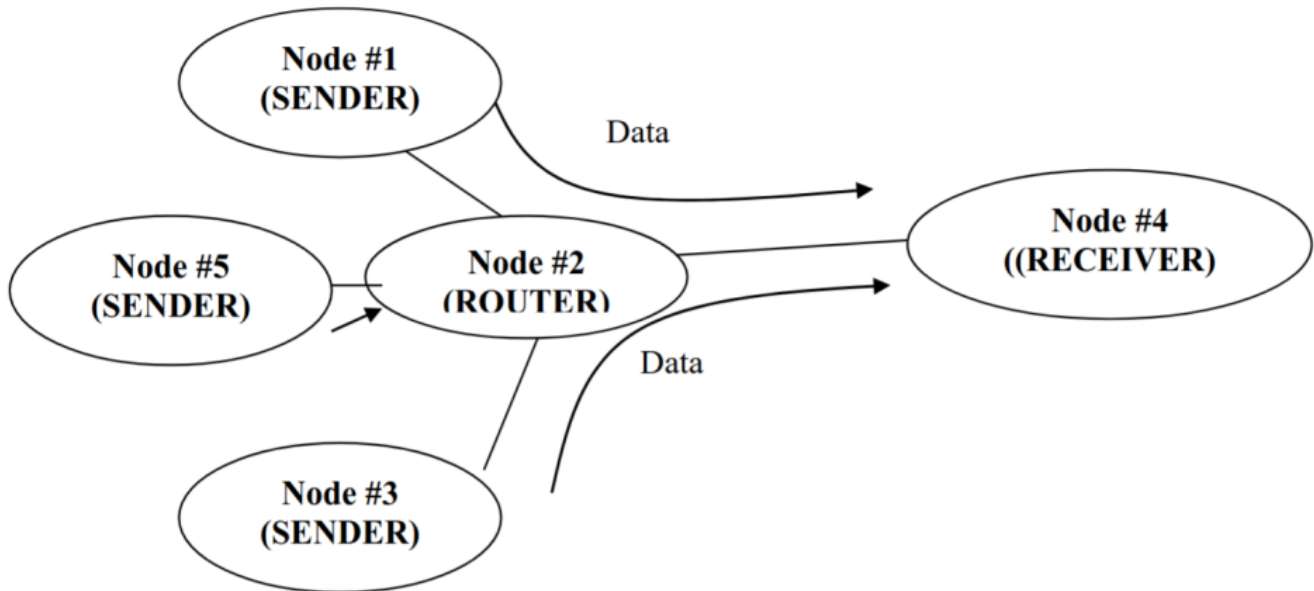
With more ACKs, the EstimatedRTT gradually stabilizes and converges to approximately **180–200 ms**, smoothing out fluctuations in SampleRTT values.

Network Simulation using NS2

Q1: Network Simulation using TCP/FTP and UDP/CBR

Problem Statement

Build a script to simulate the given network scenario:



Code

```
# Create Simulator
set ns [new Simulator]

# Define colors
$ns color 1 Blue
$ns color 2 Red
$ns color 3 Green

# Open NAM trace
set nf [open out.nam w]
$ns namtrace-all $nf

# Finish procedure
proc finish {} {
    global ns nf
    $ns flush-trace
    close $nf
    exec nam out.nam &
    exit 0
}

# Create nodes
set n0 [$ns node] ;# TCP sender 1
```

```
set n1 [$ns node] ;# Router
set n2 [$ns node] ;# TCP sender 2
set n3 [$ns node] ;# Receiver
set n4 [$ns node] ;# UDP sender

# Create links
$ns duplex-link $n0 $n1 1Mb 10ms DropTail
$ns duplex-link $n2 $n1 1Mb 10ms DropTail
$ns duplex-link $n4 $n1 1Mb 10ms DropTail
$ns duplex-link $n1 $n3 1Mb 10ms DropTail

# TCP Flow 1
set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0
set sink0 [new Agent/TCPSink]
$ns attach-agent $n3 $sink0
$ns connect $tcp0 $sink0

set ftp0 [new Application/FTP]
$ftp0 attach-agent $tcp0

# TCP Flow 2
set tcp1 [new Agent/TCP]
$ns attach-agent $n2 $tcp1
set sink1 [new Agent/TCPSink]
$ns attach-agent $n3 $sink1
$ns connect $tcp1 $sink1

set ftp1 [new Application/FTP]
$ftp1 attach-agent $tcp1

# UDP Flow
set udp0 [new Agent/UDP]
$ns attach-agent $n4 $udp0
set null0 [new Agent/Null]
$ns attach-agent $n3 $null0
$ns connect $udp0 $null0

set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0

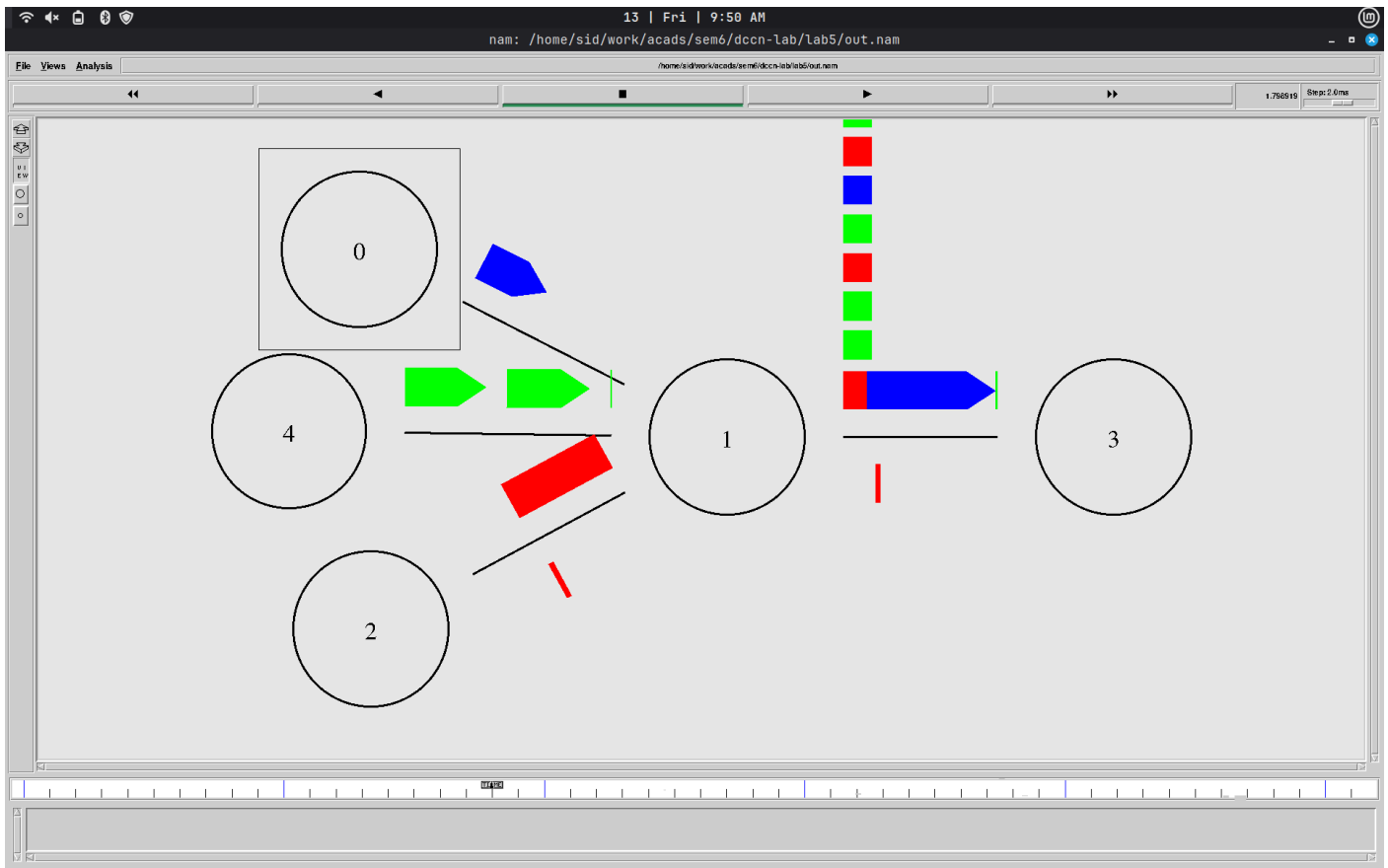
# Schedule
$ns at 0.5 "$ftp0 start"
$ns at 1.0 "$ftp1 start"
$ns at 1.5 "$cbr0 start"
$ns at 4.0 "$cbr0 stop"
$ns at 4.5 "$ftp1 stop"
$ns at 5.0 "$ftp0 stop"

# End simulation
$ns at 6.0 "finish"

# Run
```

```
$ns run
```

Output Screenshot

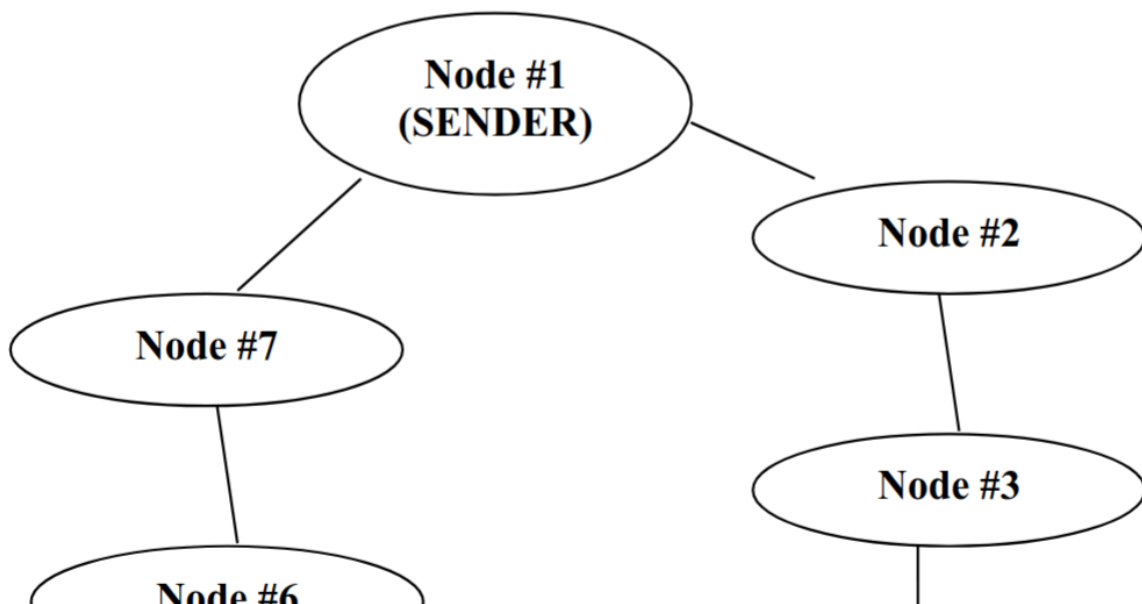


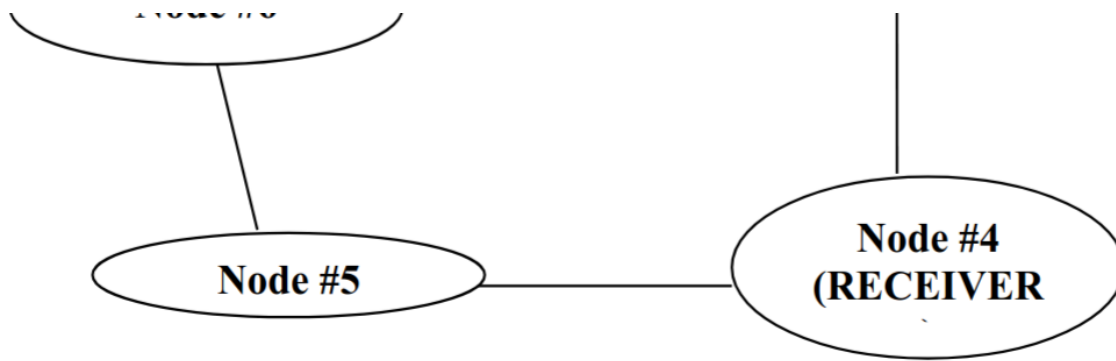
Q2: Network Simulation (Ring Topology)

Problem Statement

Build a script to simulate the given network scenario with **7 nodes in a ring topology**.

Network Diagram





Code

```
# Create Simulator
set ns [new Simulator]

# Define color
$ns color 1 Blue

# Open NAM trace
set nf [open out.nam w]
$ns namtrace-all $nf

# Finish procedure
proc finish {} {
    global ns nf
    $ns flush-trace
    close $nf
    exec nam out.nam &
    exit 0
}

# Create nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]
set n6 [$ns node]

# Create ring links
$ns duplex-link $n0 $n1 1Mb 10ms DropTail
$ns duplex-link $n1 $n2 1Mb 10ms DropTail
$ns duplex-link $n2 $n3 1Mb 10ms DropTail
$ns duplex-link $n3 $n4 1Mb 10ms DropTail
$ns duplex-link $n4 $n5 1Mb 10ms DropTail
$ns duplex-link $n5 $n6 1Mb 10ms DropTail
$ns duplex-link $n6 $n0 1Mb 10ms DropTail

# TCP Agent
set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0
```

```

# TCP Sink
set sink0 [new Agent/TCPSink]
$ns attach-agent $n3 $sink0

# Connect
$ns connect $tcp0 $sink0

# FTP Application
set ftp0 [new Application/FTP]
$ftp0 attach-agent $tcp0

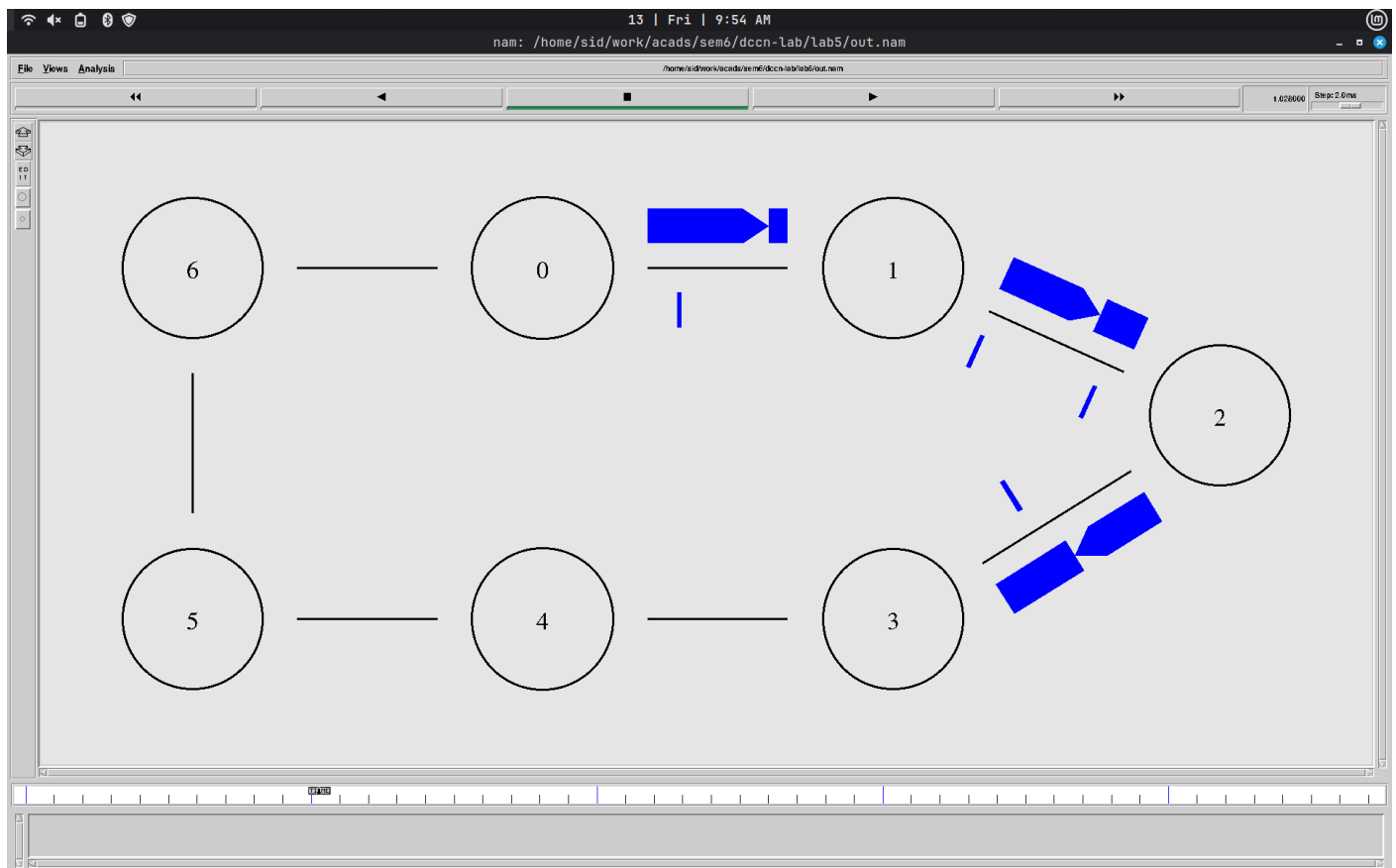
# Schedule
$ns at 0.5 "$ftp0 start"
$ns at 4.5 "$ftp0 stop"

# End simulation
$ns at 5.0 "finish"

# Run
$ns run

```

Output Screenshot



TCP Reno v/s TCP BIC

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int steps = 1200;
    double loss_prob = 0.05;

    double cwnd_reno = 1;
    double cwnd_bic = 1;

    double bic_max = 1;
    double bic_min = 1;

    ofstream fout("output.dat");

    for (int t = 1; t <= steps; t++) {

        // random loss
        bool loss = ((double)rand() / RAND_MAX) < loss_prob;

        // reno
        if (loss) {
            cwnd_reno = max(1.0, cwnd_reno / 2.0);
        } else {
            cwnd_reno += 1.0 / cwnd_reno; // linear growth
        }

        // bic
        if (loss) {
            bic_max = cwnd_bic;
            cwnd_bic = max(1.0, cwnd_bic / 2.0);
            bic_min = cwnd_bic;
        } else {
            // binary search phase
            double target = (bic_max + bic_min) / 2.0;

            if (abs(cwnd_bic - target) < 0.1) {
                cwnd_bic += 1; // aggressive increase
            } else {
                cwnd_bic += (target - cwnd_bic) * 0.1;
            }
        }

        fout << t << " " << cwnd_reno << " " << cwnd_bic << "\n";
    }

    fout.close();
    return 0;
}
```

